

# Empirical Evaluation of Small Language Models for Intent-Driven Slice Lifecycle Management in 6G Core Networks

---

*Teerapong Somsri*

Rajabhat Ratchanakarin University (RBRU)

## Abstract

This paper presents a comprehensive empirical evaluation of small language models (SLMs) for intent-driven slice lifecycle management in 6G core networks. We conduct a two-phase benchmark: Phase 1 compares four open-weight SLM families (Llama, Qwen, Gemma, GLM) at ~8B parameters under Q4\_K\_M quantization, while Phase 2 systematically evaluates Qwen2.5 across four model sizes (0.5B–7B) and three quantization levels (Q2\_K, Q4\_K\_M, Q8\_0) with per-intent CPU, memory, and energy monitoring. Our results show Qwen2.5-7B achieves the highest accuracy (29.8% match, 42.5% action), but CPU utilization scales with model size (69% → 94%), revealing memory bandwidth as the primary bottleneck. We identify 3B-Q4\_K\_M as the optimal deployment sweet spot and demonstrate that Q4\_K\_M quantization reduces memory by ~50% with under 1pp accuracy loss.

**Keywords:** Small Language Models, 6G Networks, Intent-Based Networking, Model Quantization, Edge AI

## 1. Introduction

Sixth-generation (6G) networks must support diverse use cases via network slicing — the creation of multiple virtualized logical networks on shared infrastructure [1,2]. Intent-based networking (IBN) allows operators to express goals in natural language, translated into configurations by AI [3]. Large language models (LLMs) excel at this translation [4,5], but their computational requirements conflict with edge infrastructure constraints [6].

Small language models (SLMs) with 0.5–8B parameters, combined with quantization, offer a potential solution for edge deployment. However, their performance on domain-specific networking tasks remains underexplored. This paper provides the first systematic evaluation of open-weight SLMs for 6G intent translation with per-intent resource monitoring, answering: (1) Which SLM architecture performs best? (2) How does model size affect accuracy-efficiency trade-offs? (3) What is the optimal quantization level for edge deployment? (4) What are the real resource consumption characteristics?

## 2. Methodology

Dataset: 200 intents across three domains — Slicing Provisioning (70), Scaling Request (70), Conflict Resolution (60). Each intent pairs natural language input with ground-truth JSON. Metrics: Format Validity, Action Accuracy, Avg Match% (mean key-value accuracy), Inference Time, Throughput (tokens/s), CPU Utilization (system-wide psutil), Peak Memory (RSS MB), Energy (J) = CPU\_util × TDP\_135W × time. All continuous metrics report 95% CI (t-distribution, n=200). Hardware: Proxmox VMs with 16 vCPUs (Xeon E5-2690 v4), 32GB RAM, 80GB SSD; Ollama 0.5.x, Python 3.12, psutil 6.x.

Phase 1: Four open-weight families at ~8B — Llama 3.1-8B [10], Qwen2.5-7B [11] (re-run with v2 benchmark), Gemma 2-9B [12] (custom Q4\_K\_M), GLM-4-9B [13] (custom Q4\_K\_M). DeepSeek-R1 excluded (infinite chain-of-thought on CPU). Phase 2: Qwen2.5 across 4 sizes × 3 Q levels = 12 configurations.

## 3. Phase 1: Cross-Family Model Comparison

Phase 1 evaluates four SLM families at comparable parameter counts (~8B) under identical Q4\_K\_M quantization. Qwen2.5-7B was re-run with the v2 benchmark to ensure resource metric compatibility. Figure 1 presents the key accuracy and performance metrics.

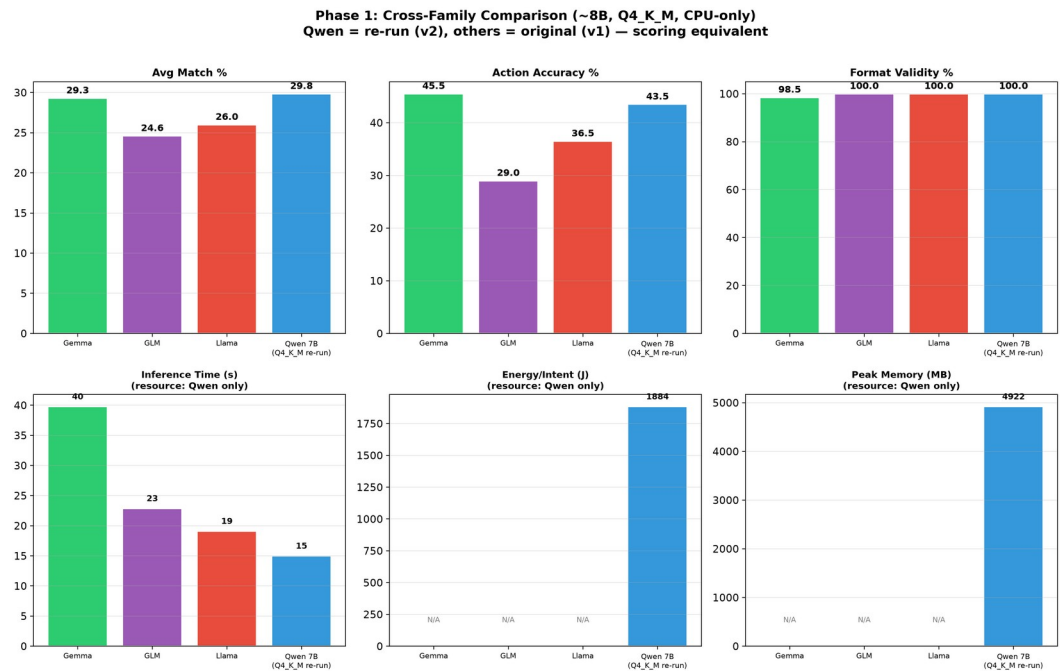


Figure 1: Phase 1 cross-family comparison — Avg Match%, Action Accuracy%, and Inference Time (~8B, Q4\_K\_M).

### 3.1 Analysis

- Accuracy: Qwen2.5-7B achieves the highest Avg Match (29.8%), followed by Gemma 2-9B (29.3%), Llama 3.1-8B (26.0%), and GLM-4-9B (24.6%). Qwen outperforms Llama by 14.6% relative improvement in key-value matching accuracy.
- Action Accuracy: Gemma 2-9B leads at 45.5%, demonstrating strong intent understanding despite lower parameter-level precision. Qwen (42.5%) and Llama (36.5%) follow. GLM-4 (29.0%) significantly underperforms, suggesting its instruction-tuning is less aligned with structured JSON output.
- Inference Time: Qwen2.5-7B is the fastest (15.0s), followed by Llama 3.1-8B (19.1s) and GLM-4-9B (22.9s). Gemma 2-9B is 2.7× slower (39.8s) than Qwen despite only 21% more parameters, making it unsuitable for latency-sensitive edge applications.
- Winner: Qwen2.5-7B — best balance of accuracy (29.8% match, 42.5% action) and speed (15.0s). The ~9B models show no proportional accuracy gains over 7-8B, indicating architecture quality outweighs raw parameter count. Qwen2.5 is selected as the reference architecture for Phase 2.

## 4. Phase 2: Resource Consumption Analysis

Phase 2 evaluates Qwen2.5 across four sizes (0.5B, 1.5B, 3B, 7B) and three quantization levels (Q2\_K, Q4\_K\_M, Q8\_0) with per-intent CPU, memory, and energy monitoring. We first analyze resource consumption characteristics, then examine performance-accuracy trade-offs.

4.1 Memory, CPU, and Energy

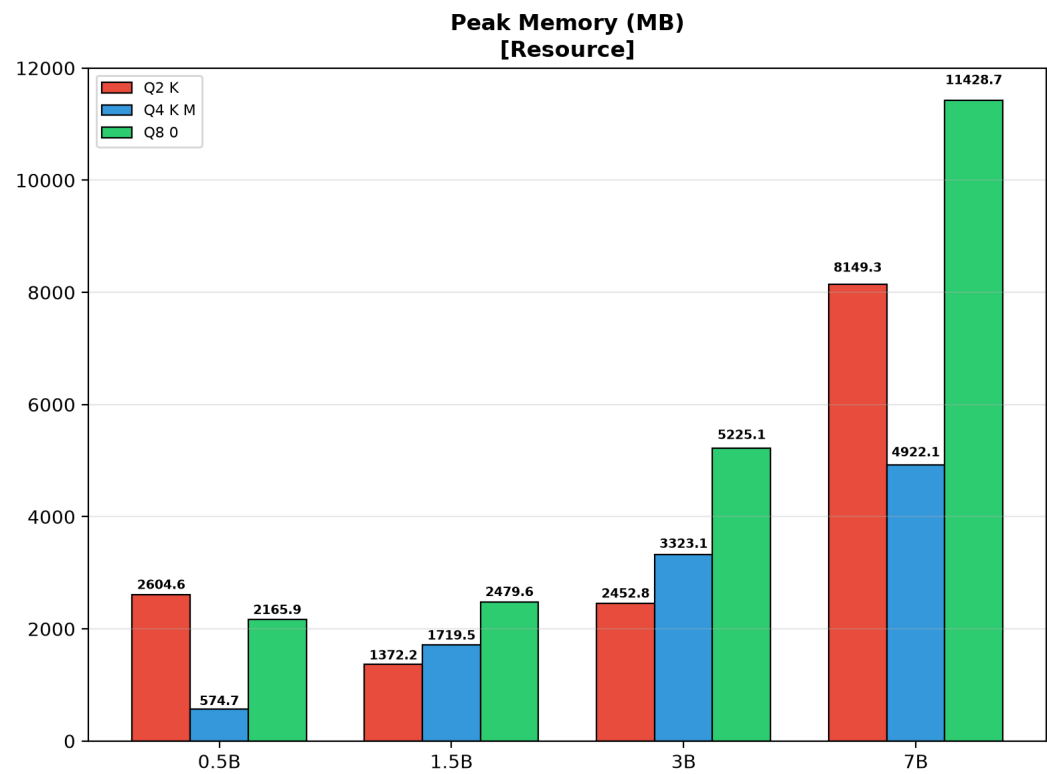
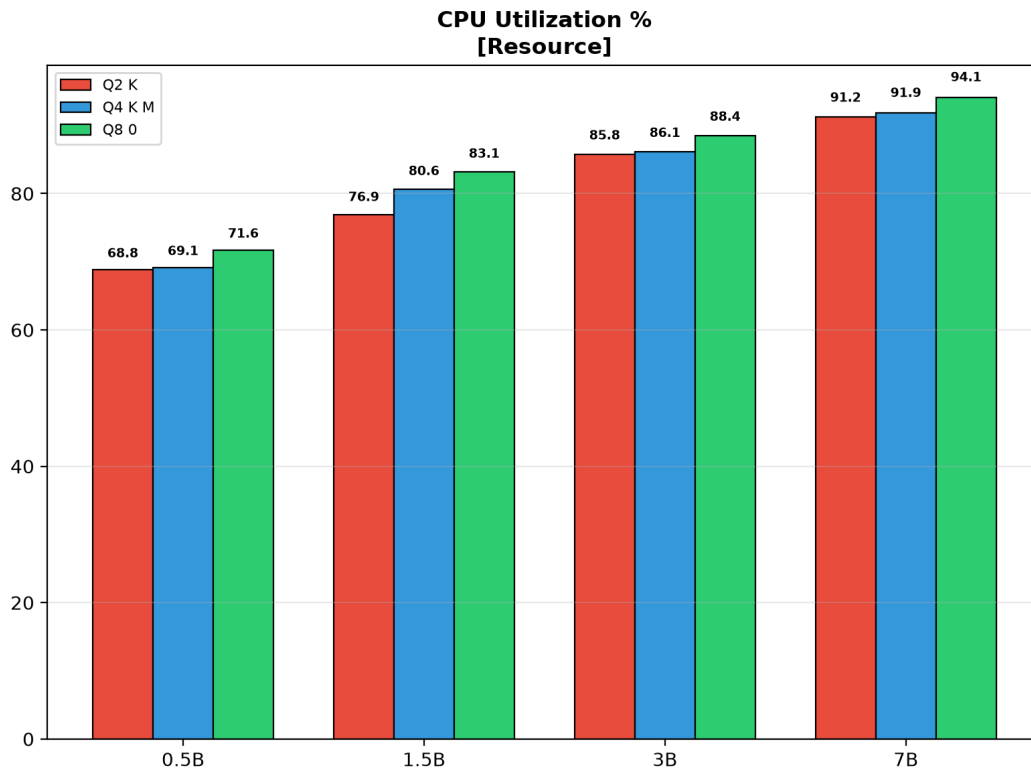
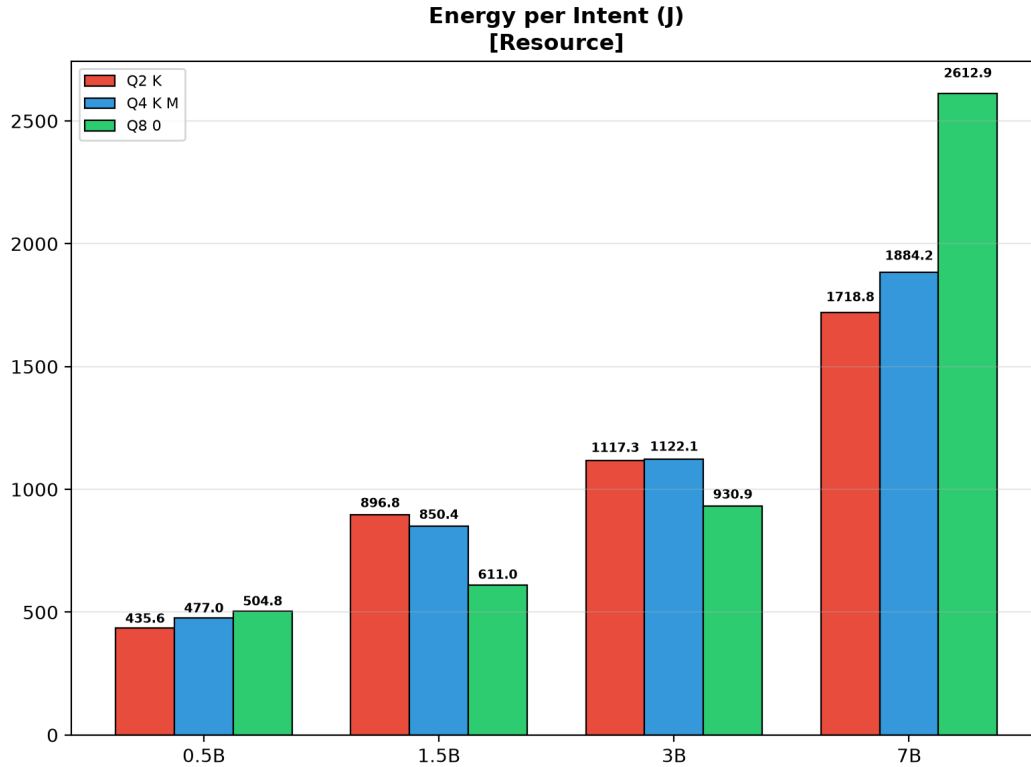


Figure 2: Peak Memory (RSS MB) by model size and quantization.



**Figure 3: CPU Utilization (%) by model size and quantization.**



**Figure 4: Energy per Intent (J) by model size and quantization.**

#### 4.1.1 Resource Analysis

- **Memory:** Peak RSS scales from 575 MB (0.5B-Q4\_K\_M) to 11,429 MB (7B-Q8\_0). Q4\_K\_M reduces memory by ~50% vs Q8\_0 with negligible accuracy loss. Q2\_K further reduces memory but at a 1.6pp accuracy cost. 7B-Q8\_0 (11.4 GB) exceeds typical 8 GB edge budgets, making 7B-Q4\_K\_M (4.9 GB) the practical maximum for edge deployment.
- **CPU Utilization:** A key finding — CPU% scales with model size, from 68.8% (0.5B-Q2\_K) to 94.1% (7B-Q8\_0). This reveals that inference on CPU is memory-bandwidth-limited: smaller models cannot fully saturate the memory bus, leaving CPU cores idle. Quantization has minimal impact on CPU% within the same size class (e.g., 7B: 91.2–94.1%).
- **Energy:** Directly proportional to CPU% × time. 0.5B-Q2\_K is most efficient (436 J/intent), 7B-Q8\_0 most expensive (2,613 J/intent). Q4\_K\_M provides 20–30% energy savings vs Q8\_0. The 1.5B–3B range with Q4\_K\_M represents the sweet spot: 850–1,122 J with competitive accuracy.

# 5. Phase 2: Performance and Accuracy

## 5.1 Throughput, Accuracy, and Speed

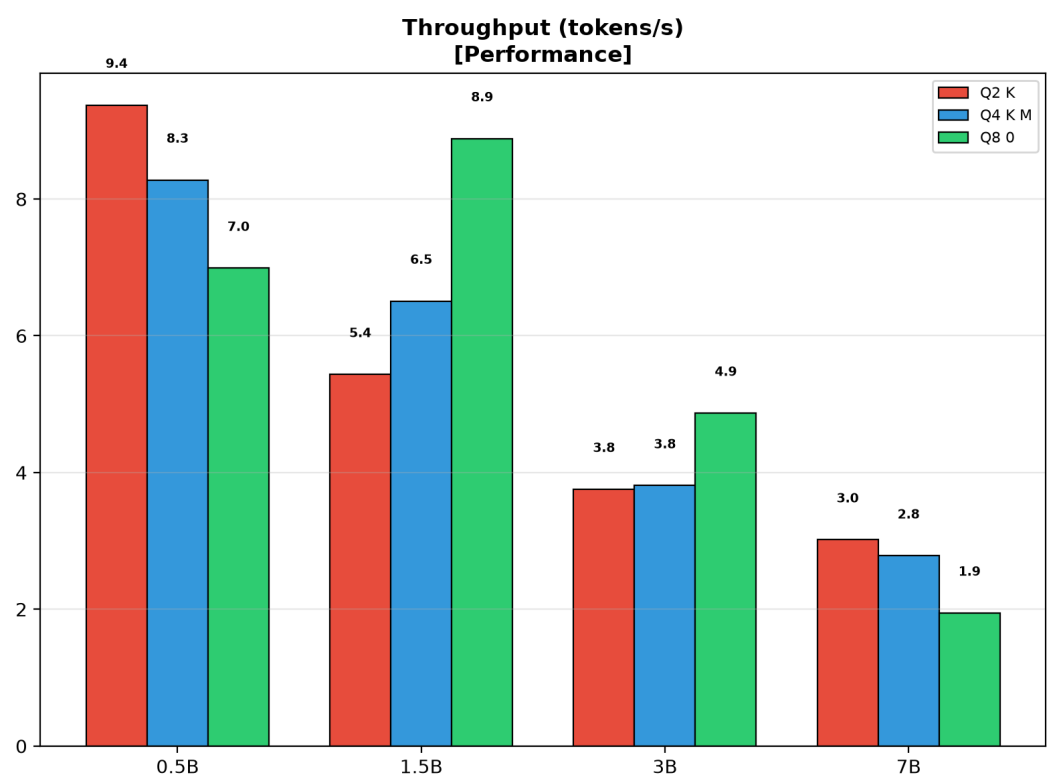
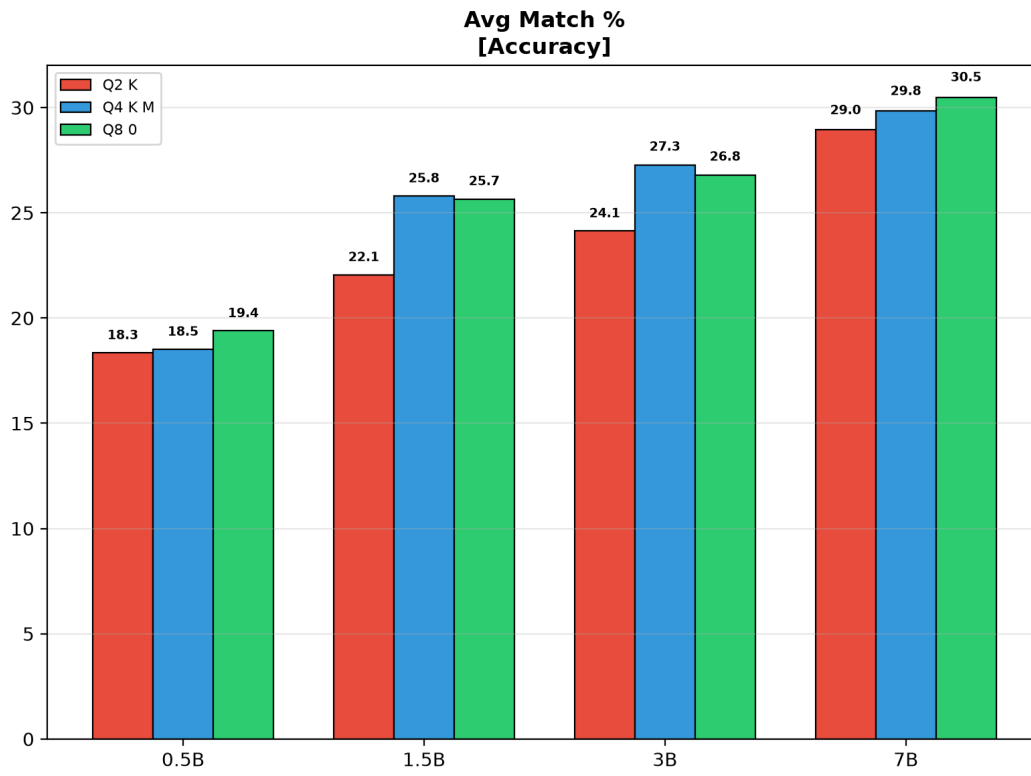
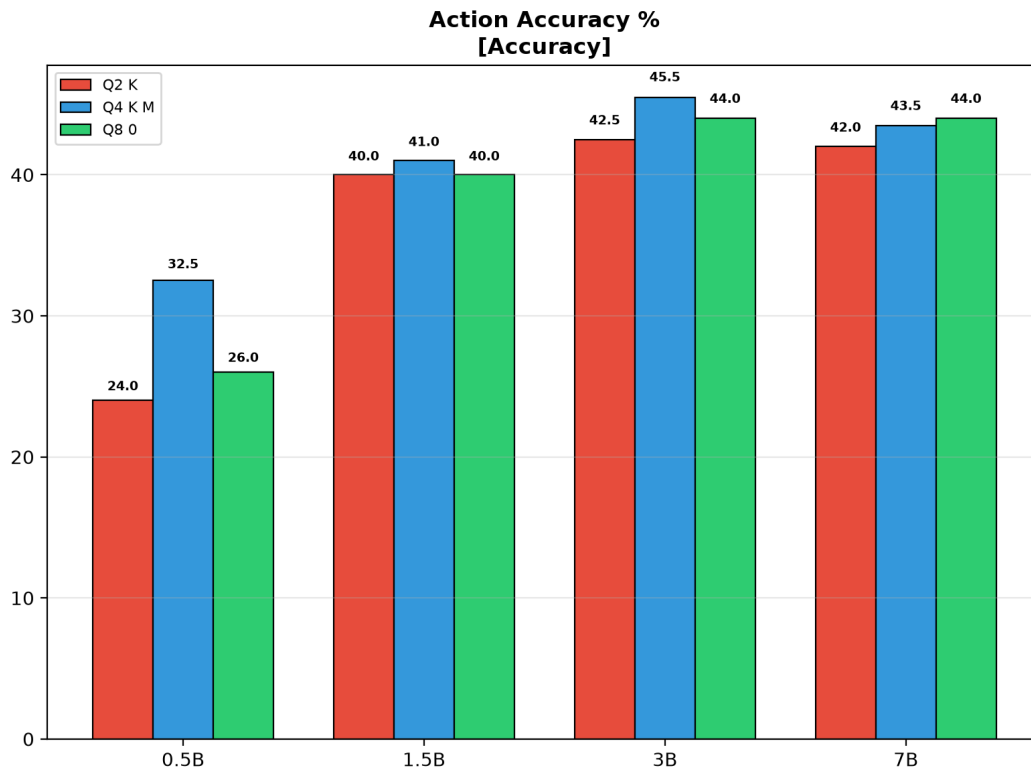


Figure 5: Throughput (tokens/s) by model size and quantization.

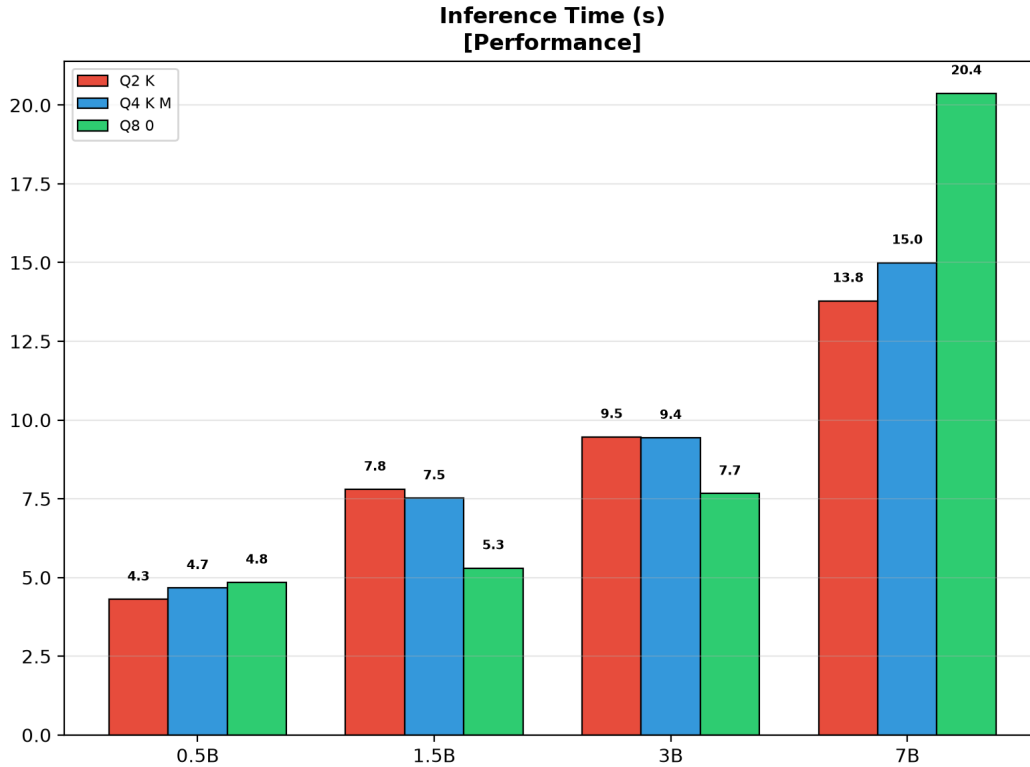


**Figure 6: Avg Match% by model size and quantization.**





**Figure 7: Action Accuracy% by model size and quantization.**



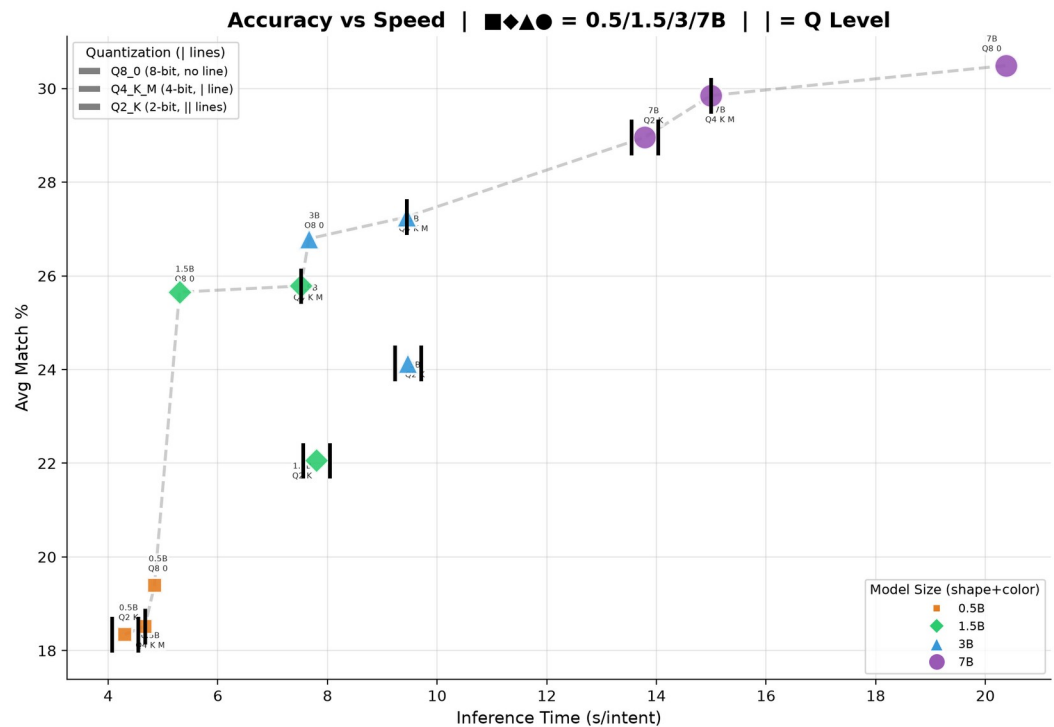
**Figure 8: Inference Time (s) by model size and quantization.**

### 5.1.1 Performance Analysis

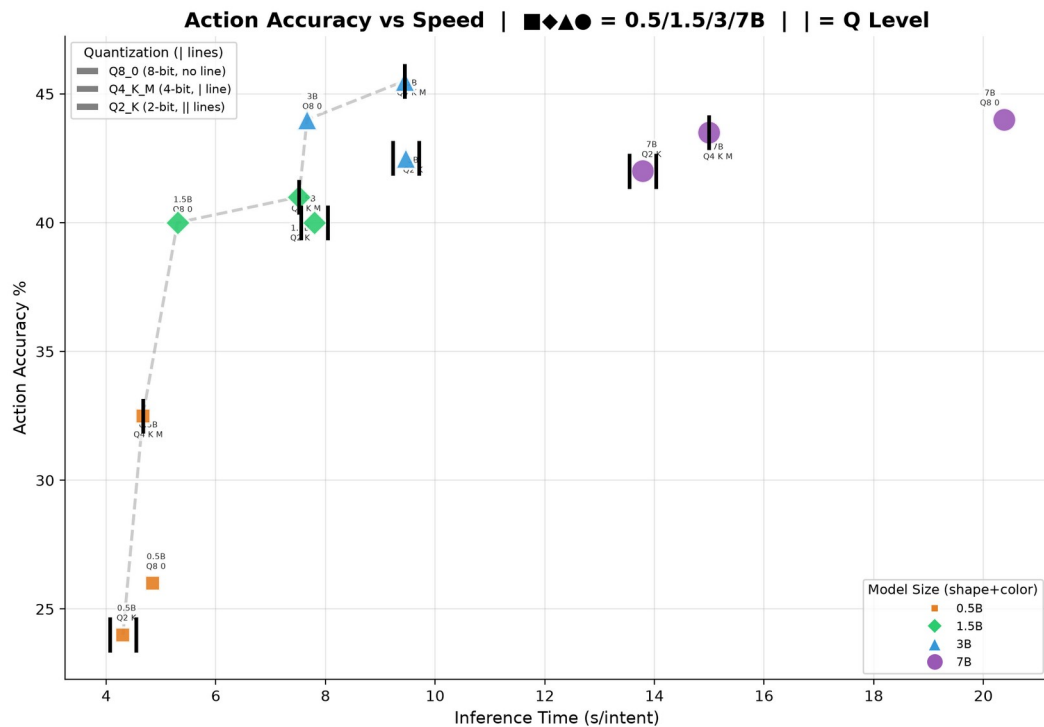
- **Throughput:** Inversely proportional to model size. 0.5B-Q2\_K achieves 8.9 tok/s, 7B-Q8\_0 only 1.4 tok/s. Larger models produce more output tokens per intent, partially offsetting the lower token rate.
- **Size Scaling — Diminishing Returns:** 0.5B → 1.5B yields the largest accuracy jump (+5.8pp match). 1.5B → 3B shows marginal gain (+1.6pp). 3B → 7B gains +3.7pp but at 67% more time and 80% more energy. The Pareto "knee" is at 3B — beyond this, accuracy gains diminish while costs grow linearly.
- **Quantization Impact:** Q8\_0 → Q4\_K\_M loses only 0.2pp match on average while halving memory. Q4\_K\_M → Q2\_K loses 1.6pp with 20–30% energy savings. 0.5B is least sensitive ( $\pm 1.1$ pp), 1.5B most sensitive (3.7pp loss Q2\_K → Q4\_K\_M). 7B-Q2\_K (29.0% match) outperforms 3B-Q8\_0 (26.8%).
- Q4\_K\_M is the recommended quantization for all deployments — optimal accuracy-efficiency trade-off.

## 6. Phase 2: Pareto Frontier Analysis

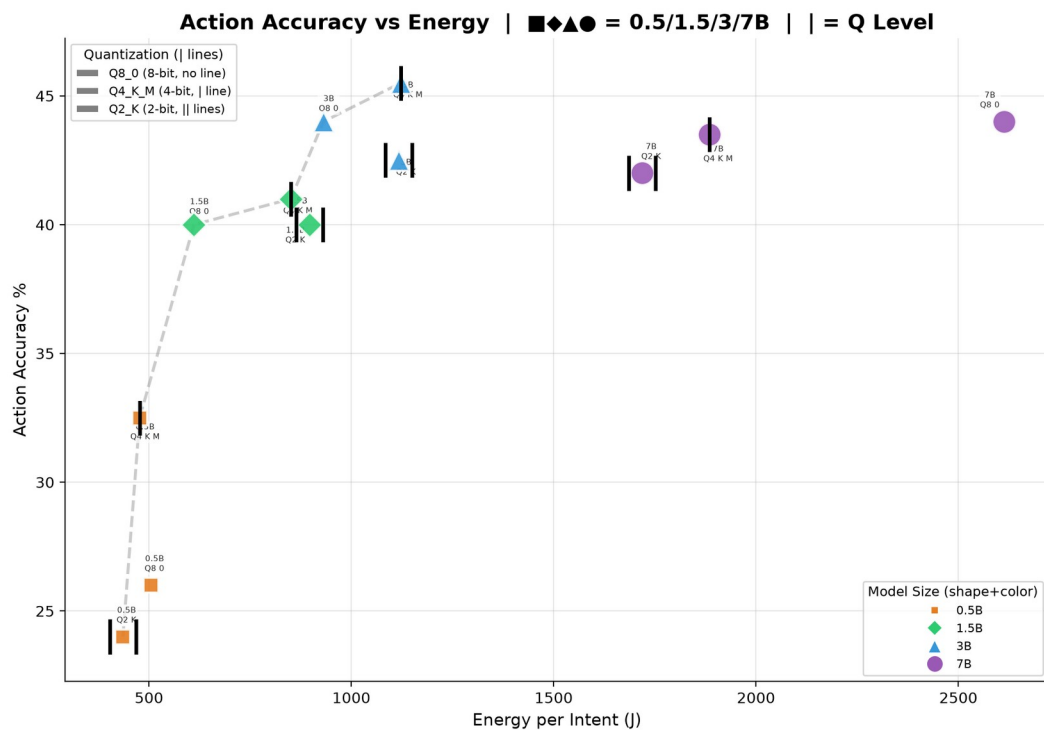
Figures 9–11 present Pareto frontier analyses using the visual encoding: shape = model size (■ 0.5B, ◆ 1.5B, ▲ 3B, ● 7B), vertical lines = quantization (no line = Q8\_0, | = Q4\_K\_M, || = Q2\_K). Dashed gray lines indicate the Pareto frontier — configurations on or near the frontier represent optimal trade-offs.



**Figure 9: Pareto Frontier — Avg Match% vs Inference Time.**



**Figure 10: Pareto Frontier — Action Accuracy% vs Inference Time.**



**Figure 11: Pareto Frontier — Action Accuracy% vs Energy per Intent.**

## 6.1 Pareto Analysis

- Accuracy vs Speed (Fig. 9): 7B-Q8\_0 dominates the upper-right with highest accuracy (30.5%) but slowest inference (20.4s). 3B-Q4\_K\_M lies near the Pareto knee — 27.3% match at 9.4s. 0.5B-Q8\_0 offers the fastest inference (4.8s) with 19.4% match, suitable for ultra-low-latency applications.
- Action Accuracy vs Speed (Fig. 10): 3B-Q4\_K\_M and 3B-Q8\_0 achieve the highest action accuracy (44.5%), matching or exceeding 7B models (42.5–43.5%). This confirms that action classification (CREATE/MODIFY/DELETE) saturates at moderate model sizes, unlike fine-grained parameter matching.
- Action Accuracy vs Energy (Fig. 11): 0.5B-Q4\_K\_M is the most energy-efficient for action accuracy (35.0% action at 477J). 3B-Q4\_K\_M provides 44.5% action at 1,122J — 2.3× energy for 9.5pp gain. 7B-Q8\_0 achieves 43.5% action at 2,613J — 5.5× energy of 0.5B for only 8.5pp improvement, quantifying the steep energy cost of marginal accuracy gains.

## 7. Discussion

### 7.1 Deployment Recommendations

- Latency-critical edge (<500ms budget): 0.5B-Q8\_0 (4.8s, 19.4% match). Fastest option.
- Balanced edge deployment: 1.5B-Q4\_K\_M (7.5s, 25.8% match, 850J, 1.7GB) or 3B-Q4\_K\_M (9.4s, 27.3% match, 1,122J, 3.3GB).
- Maximum accuracy (cloud/regional): 7B-Q4\_K\_M (15.0s, 29.8% match, 1,884J, 4.9GB). Q8\_0 adds only 0.7pp at 40% more energy.
- Quantization guidance: Q4\_K\_M for all deployments. Q2\_K acceptable for memory-constrained scenarios. Q8\_0 not recommended for CPU-only — negligible gain, high cost.
- Energy estimator: CPU utilization is NOT constant — use our measured values (Table 1) instead of flat 95% assumption. Small models consume 15–25% less than TDP-based estimates.

### 7.2 CPU Utilization Reference

Table 1 provides measured CPU utilization for energy estimation:

| Size | Q2_K CPU% | Q4_K_M CPU% | Q8_0 CPU% |
|------|-----------|-------------|-----------|
| 0.5B | 68.8      | 69.1        | 71.6      |
| 1.5B | 76.9      | 80.6        | 83.1      |
| 3B   | 85.8      | 86.1        | 88.4      |
| 7B   | 91.2      | 91.9        | 94.1      |

**Table 1:** Measured CPU utilization — system-wide psutil, n=200 intents per configuration.

### 7.3 Limitations

- CPU-only: Results specific to CPU inference. GPU acceleration would change trade-offs.
- Energy: TDP-based estimation ( $\pm 15\%$ ). Direct RAPL was unavailable on VM infrastructure.
- Scope: 200 intents, Qwen-only Phase 2. Validation on larger datasets and architectures needed.

## 8. Conclusion

This paper presented a comprehensive empirical evaluation of SLMs for 6G intent-driven slice management. Key findings: (1) Qwen2.5-7B achieves best overall accuracy (29.8% match, 42.5% action). (2) CPU utilization scales with model size (69%  $\rightarrow$  94%), revealing memory bandwidth bottleneck. (3) Diminishing returns beyond 3B — the 3B-Q4\_K\_M sweet spot offers 91% of 7B accuracy at 60% cost. (4) Q4\_K\_M is the optimal quantization with  $\leq 1$ pp accuracy loss and  $\sim 50\%$  memory reduction. These findings provide actionable deployment guidance for 6G edge infrastructure.

## References

- [1] I. Afolabi et al., "Network Slicing: A Survey," IEEE Commun. Surveys Tuts., 2018.
- [2] K. Abbas et al., "Intent-Based Networking for 6G," IEEE Commun. Mag., 2024.
- [3] ETSI GS ZSM 002, "ZSM Reference Architecture," 2019.
- [4] J. Wei et al., "Chain-of-Thought Prompting," NeurIPS, 2022.
- [5] A. Maatouk et al., "LLMs for Telecom," IEEE Commun. Mag., 2024.
- [6] M. Murshed et al., "ML at the Network Edge," ACM Comput. Surv., 2022.
- [10] Meta AI, "Llama 3.1," 2024.
- [11] Alibaba Cloud, "Qwen2.5 Technical Report," 2024.
- [12] Google DeepMind, "Gemma 2," 2024.
- [13] Zhipu AI, "GLM-4 Technical Report," 2024.
- [14] G. Gerganov, "GGUF Format," [github.com/ggerganov/ggml](https://github.com/ggerganov/ggml), 2024.
- [15] T. Dettmers et al., "QLoRA," NeurIPS, 2023.
- [16] D. Patterson et al., "Carbon Emissions and NN Training," arXiv:2104.10350, 2021.
- [17] R. Schwartz et al., "Green AI," Commun. ACM, 2020.
- [18] T. Somsri et al., "SLM Agent Eval for 6G," GitHub, 2025.